

BOOLEAN ALGEBRA

CHAPTER

Learning Objective

At the end of the chapter students will:

- Learn Fundamental concepts and basic laws of Boolean algebra.
- Learn about Boolean expression and will be able to generate same.
- Understand the basic operations used in computers and other digital circuits.
- Be able to represent Boolean logic problems as - truth table
- Understand how logic relates to computing problems.
- Handle simple Boolean function in SOP and POS form
- Apply rules to simplify /expand Boolean terms / functions
- Simplify the Boolean function using K map and algebraic method
- Establish the correspondence between Karnaugh maps, truth table and logical expressions.
- Learn about gates that process logic signals.
- Understand basic terminology, type of logic gates to Learn about logic circuits and Boolean expression
- Learn how to design small circuits
- Learn to determine output of the gate(s) / circuit for the given input values.
- Learn about universality of NAND & NOR gate
- Explore the application of Boolean algebra in the design of electronic circuits

INTRODUCTION

Boolean algebra was introduced by George Boole in 1854.

Boolean Algebra is made up of

- **Elements** – These are boolean variables or constants with value 1 or 0.

- **Truth Table**
- **Operators - And, Or and Not.**
 - AND is a binary operator, to perform logical multiplication, it is represented by '.'
 - OR is also a binary operator, to perform logical addition. It is represented by '+'. It is represented by '+'.
 - NOT is a unary operator, to complement the operand. Not is represented as ' ' or '-'.

Complement is the inverse of a variable/ constant.
- **Axioms and Theorems.**

TRUTH TABLE

A truth table is a chart of 1's and 0's, arranged to indicate the results of all possible input options. It provides the basic method of describing a Boolean expression.

TRUTH TABLE: ONE VARIABLE

Let us consider a boolean variable **a**. There can be only two possible input combination of one variable ie 1 or 0. The Truth Table for this boolean variable is:

a
0
1

TRUTH TABLE: TWO VARIABLES

Let us consider two boolean variables **a** and **b**. There are four possible input combinations or 2^2 input combinations. The Truth Table for these boolean variables is:

a	b
0	0
0	1
1	0
1	1

TRUTH TABLE: THREE VARIABLES

Let us consider three boolean variables **a**, **b** and **c**. There are eight possible input combinations or 2^3 input combinations. The Truth Table for these boolean variables is:

a	b	c
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

A 4-input variable would have 16 or 2^4 combinations and so on as the number of inputs increases.

a	b	c	d
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	0	1
1	1	1	0
1	1	1	1

OPERATORS

There are three basic operations in Boolean algebra:

- **NOT**

It is a unary operator used to complement the operand. It is represented by ' or $\bar{}$. If x is a boolean variable then NOT x is represented by x' . The truth table for this is:

x	x'
0	1
1	0

Second column of table gives the value of x' . This table tells that if x is true, x' will be false and vice versa, as is evident from 2nd column of table.

- **AND, OR**

They are binary operators and represent logical multiplication and addition respectively. AND is represented by the symbol '+' and OR is represented by the symbol ' $\cdot$ '. Truth table for the boolean function applied on two variables x and y with Boolean operators AND and OR is:

x	y	$x \cdot y$	$x + y$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

AXIOMS AND THEOREMS

Identity	$A + 0 = A$	$A \cdot 1 = A$
Complement	$A + A' = 1$	$A \cdot A' = 0$
Commutative	$A + B = B + A$	$A \cdot B = B \cdot A$
Associative	$A + (B + C) = (A + B) + C$	$A \cdot (B \cdot C) = (A \cdot B) \cdot C$
Distributive	$A \cdot (B + C) = A \cdot B + A \cdot C$	$A + (B \cdot C) = (A + B) \cdot (A + C)$

Null Element	$A + 1 = 1$	$A \cdot 0 = 0$
Involution	$(A')' = A$	
Idempotent	$A + A = A$	$A \cdot A = A$
Absorption	$A + (A \cdot B) = A$	$A \cdot (A + B) = A$
	$A + A' \cdot B = A + B$	$A \cdot (A' + B) = A \cdot B$
	$(A+B) \cdot (A'+C) \cdot (B+C) = (A+B) \cdot (A'+C)$	$(A \cdot B) + (A' \cdot C) + (B \cdot C) = A \cdot B + A' \cdot C$
De Morgan's	$(A + B)' = A' \cdot B'$	$(A \cdot B)' = A' + B'$

DUALITY PRINCIPLE

Duality Principle states that starting with Boolean relation, you can device another Boolean relation by

1. Changing each OR operation to AND
2. Changing each AND operation to OR
3. Complementing the identity (0 or 1) elements.
4. Not changing the form of variable

For example:

dual of relation $x + 1 = 1$ is $x \cdot 0 = 0$.

The second part of axioms and theorems in the above table is actually the result of this principle only.

PROOF OF AXIOMS AND THEOREMS

Theorems/laws of Boolean Algebra can be proved in two ways:

- Algebraic Method:** In this method, we use axioms & theorems.
- Truth Table:** Here we use truth table to show that expression given on LHS results into same values for expression on RHS. This actually is verification of theorem, not proving.

ASSOCIATIVE LAW:

Theorem states that $A + (B+C) = (A+B) + C$ and $A.(B.C) = (A.B).C$

Truth table method:

A	B	C	B+C	LHS: A+(B+C)	(A+B)	RHS: (A+B)+C
0	0	0	0	0	0	0
0	0	1	1	1	0	1
0	1	0	1	1	1	1
0	1	1	1	1	1	1
1	0	0	0	1	1	1
1	0	1	1	1	1	1
1	1	0	1	1	1	1
1	1	1	1	1	1	1

As LHS= RHS, hence verified

By the Duality principle we can say that second form of the theorem will also be true.

NULL ELEMENT

It states that $A + 1 = 1$ and $A . 0 = 0$

Truth Table method:

A	1	LHS: A+1	RHS:1
0	1	1	1
1	1	1	1

As LHS= RHS, hence verified

Algebraic method:

Taking LHS:

$$\begin{aligned}
 A+1 &= (A+1).1 && \text{by Identity} \\
 &= (A+1).(A+A') && \text{by Complement} \\
 &= A + (1.A') && \text{by Distribution} \\
 &= A + (A'.1) && \text{By Idempotent \& Commutative} \\
 &= A+A' && \text{by Identity} \\
 &= 1 && \text{by Complement}
 \end{aligned}$$

By applying the principle of Duality, we get $A \cdot 0 = 0$

LAW OF INVOLUTION

It states that $((A'))' = A$

Algebraic method:

Taking LHS

$$\begin{aligned}(A')' &= (A')' + 0 && \text{by Identity} \\ &= (A')' + (A' \cdot A) && \text{by Complement} \\ &= ((A')' + A') \cdot ((A')' + A) && \text{by Distribution} \\ &= 1 \cdot ((A')' + A) && \text{by Complement} \\ &= (A')' + A && \text{by Identity} \quad (1)\end{aligned}$$

Taking LHS again

$$\begin{aligned}(A')' &= (A')' \cdot 1 && \text{by Identity} \\ &= (A')' \cdot (A + A') && \text{by Complement} \\ &= (A')' \cdot A + ((A')' \cdot A') && \text{by Distribution} \\ &= (A')' \cdot A + 0 && \text{by Complement} \\ &= (A')' \cdot A && (2)\end{aligned}$$

Substituting the value of $(A')' = (A')' \cdot A$ in (1), we get

$$\begin{aligned}(A')' + A &= (A')' \cdot A + A \\ &= A + (A')' \cdot A && \text{by Commutative} \\ &= A + A \cdot (A')' && \text{by Commutative} \\ &= A && \text{by Absorption}\end{aligned}$$

By the Duality principle we can say that second form of the theorem will also be true.

Truth Table method:

A	(A')	LHS: (A')'	RHS: A
0	1	0	0
1	0	1	1

As LHS = RHS, hence verified

IDEMPOTENT LAW

It states that $A+A=A$ and $A \cdot A = A$

Algebraic method:

Taking L.H.S

$$\begin{aligned} A+A &= (A+A).1 && \text{by identity} \\ &= (A+A). (A+A') && \text{by complement} \\ &= (AA+AA') + (AA + AA') && \text{by distribution} \\ &= AA+0 \\ &= A \end{aligned}$$

By the Duality principle we can say that second form of the theorem will also be true.

Truth Table method:

A	RHS: A	LHS: A+A
0	0	0
1	1	1

As LHS = RHS, hence verified

ABSORPTION LAW

i) $A + AB = A$ and $A \cdot (A + B) = A$

Algebraic method:

Taking LHS

$$\begin{aligned} A + AB &= (A.1) + (A.B) && \text{by Identity} \\ &= A. (1+B) && \text{by Distribution} \\ &= A.1 && \text{by Null Element} \\ &= A \end{aligned}$$

By the Duality principle we can say that second form of the theorem will also be true.

Truth Table method:

A	B	A.B	LHS: A+AB	RHS: A
0	0	0	0	0
0	1	0	0	0

1	0	0	1	1
1	1	1	1	1

As LHS= RHS, hence verified

ii) $(A + A'B) = A + B$ and $A \cdot (A' + B) = A \cdot B$

Algebraic method:

Taking LHS

$$\begin{aligned}
 A + A'B &= (A + A') \cdot (A + B) && \text{by Distribution} \\
 &= 1 \cdot (A + B) && \text{by Complement} \\
 &= A + B && \text{by Identity}
 \end{aligned}$$

Because of duality, $A \cdot (A' + B) = A \cdot B$ is also true.

Truth Table method:

A	B	A'B	LHS: A+A'B	RHS: A+B
0	0	0	0	0
0	1	1	1	1
1	0	0	1	1
1	1	0	1	1

As LHS= RHS, hence verified

iii) $((A+B) \cdot (A'+C) \cdot (B+C)) = (A+B) \cdot (A'+C)$ AND $((A \cdot B) + (A' \cdot C) + (B \cdot C)) = A \cdot B + A' \cdot C$

Algebraic method:

Taking LHS

$$\begin{aligned}
 &= (A+B) \cdot (A'+C) \cdot (B+C) \\
 &= (A+B) \cdot (A'+C) \cdot (B+C) \cdot (B+C) && \text{by Idempotent} \\
 &= (A+B) \cdot (B+C) \cdot (A'+C) \cdot (B+C) && \text{by Commutative} \\
 &= (B+A) \cdot (B+C) \cdot (C+A') \cdot (C+B) && \text{by Commutative} \\
 &= (B+A \cdot C) \cdot (C + A'B) && \text{by Distributive} \\
 &= (B+AC) \cdot C + (B+AC) \cdot A'B && \text{by}
 \end{aligned}$$

Distribution

$$= B \cdot C + A \cdot C \cdot C + B \cdot A' \cdot B + A \cdot C \cdot A'B$$

Distribution

$$\begin{aligned}
&= B.C + A.(C.C) + (B.B). A' + C. (A. A').B && \text{by Associative and Idempotent} \\
&= B.C + A.C + B.A' + 0 && \text{by Complement, Idempotent} \\
&= (B+A).C + B.A' + A. A' && \text{by Complement and Distributive} \\
&= (B+A).C + (B+A).A' && \text{by Distributive} \\
&= (B+A).(C + A') && \text{by Distributive} \\
&= (A+B). (A'+C) && \text{by Commutative}
\end{aligned}$$

By the Duality principle we can say that second form of the theorem will also be true.

Truth Table method:

A	B	C	A+B	A'+C	B+C	LHS: (A+B).(A'+C).(B+C)	RHS: (A+B). (A'+C)
0	0	0	0	1	0	0	0
0	0	1	0	1	1	0	0
0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	1
1	0	0	1	0	0	0	0
1	0	1	1	1	1	1	1
1	1	0	1	0	1	0	0
1	1	1	1	1	1	1	1

As LHS= RHS, hence verified

DE MORGAN'S THEOREM

It states that $(A+B)' = A'.B'$ and $(A.B)' = A' + B'$

Algebraic method:

We know that

$X+X' = 1$ and $X.X' = 0$ by Complementation law

So by showing that

$$\text{i) } (A+B) + A'.B' = 1 \quad \text{ii) } (A+B). (A'.B') = 0$$

We shall establish the DeMorgan's theorem.

$$\text{i) } (A+B) + A'.B' = 1$$

Taking LHS

$$\begin{aligned}
&= (A+B) + (A' \cdot B') = ((A+B) + A') \cdot ((A+B) + B') && \text{By Distribution} \\
&= (A+(B+A')) \cdot (A+(B+B')) && \text{By Associativity} \\
&= (A+(A'+B)) \cdot (A+(B+B')) && \text{By Commutativity} \\
&= (A+(A'+B)) \cdot (A+1) && \text{By Complement} \\
&= ((A+A')+B) \cdot (A+1) && \text{By Commutativity} \\
&= (1+B) \cdot (A+1) && \text{By Complement} \\
&= 1 \cdot 1 && \text{By Null element} \\
&= 1
\end{aligned}$$

ii) $(A+B) \cdot (A' \cdot B') = 0$

Taking LHS

$$\begin{aligned}
&= (A \cdot (A' \cdot B') + (B \cdot (A' \cdot B'))) \\
&= ((A \cdot A') \cdot B') + ((B \cdot A') \cdot B') && \text{By Associativity} \\
&= 0 + ((B' \cdot A') \cdot B') && \text{By Null Element} \\
&= 0 + ((A' \cdot B) \cdot B') && \text{By Commutativity} \\
&= 0 + (A' + (B \cdot B')) && \text{By Associativity} \\
&= 0 + 0 && \text{By Null Element} \\
&= 0
\end{aligned}$$

Truth Table method:

A	B	A+B	LHS: $(A+B)'$	A'	B'	RHS: $A' \cdot B'$
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

As LHS = RHS, hence verified.

Hence De Morgan's theorem is established. By the Duality principle we can say that second form of the theorem will also be true.

Derivation of Boolean Expressions:

Minterm:

A minterm is a product of all variables/literals present in the boolean expression either in direct or complement form.

RULES FOR OBTAINING MINTERMS FROM TRUTH TABLE

1. Make a truth table of all boolean variables present in the expression.
2. If the boolean variable has value 0 then complement that variable.
3. If the boolean variable has value 1 then take the direct form of that variable.
4. Finally take the product of all variables.
5. These terms are designated using **m**.

Following table provides minterm for 3 variables:

a	b	c	Minterm	Designation
0	0	0	$a'b'c'$	m_0
0	0	1	$a'b'c$	m_1
0	1	0	$a'bc'$	m_2
0	1	1	$a'bc$	m_3
1	0	0	$ab'c'$	m_4
1	0	1	$ab'c$	m_5
1	1	0	abc'	m_6
1	1	1	abc	m_7

For four variables, there will be 16 minterms designated by m_0 to m_{15} .

Maxterm:

A maxterm is a sum of all variables/literals present in the boolean expression either in direct or complement form.

RULES FOR OBTAINING MINTERMS FROM TRUTH TABLE

1. Make a truth table of all boolean variables present in the expression.
2. If the boolean variable has value 1 then complement that variable.
3. If the boolean variable has value 0 then take the direct form of that variable.
4. Finally take the sum of all variables.
5. These terms are designated using **M**.

Following table provides maxterm for 3 variables:

a	b	c	Maxterm	Designation
0	0	0	$a+b+c$	M_0
0	0	1	$a+b+c'$	M_1
0	1	0	$a+b'+c$	M_2
0	1	1	$a+b'+c'$	M_3
1	0	0	$a'+b+c$	M_4
1	0	1	$a'+b+c'$	M_5
1	1	0	$a'+b'+c$	M_6
1	1	1	$a'+b'+c$	M_7

Maxterm for four variables can be obtained in similar manner and they are designated as M_0 to M_{15} .

Note: Each maxterm is complement of its corresponding minterm.

BOOLEAN EXPRESSION

A boolean expression is made up of boolean variables that can take only two possible values 0 or 1 and are joined by boolean operators AND, OR and NOT.

Boolean expression can be present in two forms:

SOP(Sum Of Product) form – It is a boolean expression which contains sum of terms which may or may not be min terms.

For example:

$$F(x,y,z) = xy + xy'z + y'z' + xyz$$

Here xy and $y'z'$ are not minterms because they do not contain all the variables (ie x, y, z) in direct or complement form.

POS(Product Of Sum) form – It is a boolean expression which contains product of terms which may or may not be max terms.

For example:

$$F(x,y,z) = (x+y) \cdot (x+y'+z) \cdot (y'+z') \cdot (x+y+z)$$

Here $x+y$ and $y'+z'$ are not maxterms because they do not contain all the variables (ie x, y, z) in direct or complement form.

CANONICAL EXPRESSION

Boolean expression in which all the terms are either minterms or maxterms is called as canonical expression.

Canonical expression can be present in two forms:

Canonical SOP(Sum Of Product) form – It is a boolean expression which contains sum of all minterms.

For example:

$$F(x,y,z) = xyz' + xy'z + x'y'z + xyz$$

Here all terms are minterms.

Canonical POS(Product Of Sum) form – It is a boolean expression which contains product of all max terms.

For example:

$$F(x,y,z) = (x+y+z') \cdot (x+y'+z) \cdot (x+y'+z') \cdot (x+y+z)$$

Here all terms are maxterms.

Writing Boolean expression with the help of min terms:

For eg. Consider the following truth table:

a	b	c	F1	Minterm
0	0	0	0	$a'b'c'$
0	0	1	1	$a'b'c$
0	1	0	0	$a'bc'$
0	1	1	0	$a'bc$
1	0	0	1	$ab'c'$
1	0	1	0	$ab'c$
1	1	0	1	abc'
1	1	1	1	abc

Here the function F1 has value 1 for the minterms: **a'b'c**, **ab'c'**, **abc'** and **abc**, so algebraic representation of the function F1 will be

$$F1(a,b,c) = a'b'c + ab'c' + abc' + abc$$

This is a boolean expression, expressed as sum of minterms and, will give value 1.

This form of expression is known as **Sum of Products or SOP** expression. F1 in SOP form can also be represented as

i) $m_1 + m_4 + m_6 + m_7$

Instead of using Product terms, we can just mention the minterm designation.

ii) $\sum (1, 4, 6, 7)$

This one is mathematical representation of the above expression.

CONVERT NON CANONICAL SOP EXPRESSION TO CANONICAL SOP EXPRESSION

Let's take one example:

Q Convert $XY' + Y'Z'$ into canonical SOP form.

Ans Steps to solve this problem

1. For a given expression, prepare a truth table for all possible combinations of inputs.
2. Add a new column for minterms and list all the minterms for all possible combinations.
3. Add all the minterms for which the output is 1. This gives the canonical SOP form.

X	Y	Z	Y'	Z'	XY'	Y'Z'	XY' + Y'Z'
0	0	0	1	1	0	1	1
0	0	1	1	0	0	0	0
0	1	0	0	1	0	0	0
0	1	1	0	0	0	0	0
1	0	0	1	1	1	1	1
1	0	1	1	0	1	0	1
1	1	0	0	1	0	0	0
1	1	1	0	0	0	0	0

The canonical SOP expression is:

$$F(x,y,z) = x'y'z' + xy'z' + xy'z$$

OR

$$= m_0 + m_4 + m_5$$

OR

$$= \sum(0,4,5)$$

To represent the function in complement form i.e. for its value 0, we will OR the minterms, from the truth table, for which function value is 0. Thus the function in complement form is:

$$F'(x,y,z) = x'y'z + x'yz' + x'yz + xyz' + xyz \quad (\text{the remaining minterms})$$

Writing Boolean expression with the help of max terms

For eg. Consider the following truth table

a	b	c	F1	Maxterm
0	0	0	0	$a+b+c$
0	0	1	1	$a+b+c'$
0	1	0	0	$a+b'+c$
0	1	1	0	$a+b'+c'$
1	0	0	1	$a'+b+c$
1	0	1	0	$a'+b+c'$
1	1	0	1	$a'+b'+c$
1	1	1	1	$a'+b'+c'$

Here the function F1 has value 0 for the maxterms: $a+b+c$, $a+b'+c$, $a+b'+c'$ and $a'+b+c'$, so algebraic representation of the function F1 will be

$$F1(a,b,c) = (a+b+c) \cdot (a+b'+c) \cdot (a+b'+c') \cdot (a'+b+c')$$

This is a boolean expression, expressed as sum of maxterms and, will give value 0.

This form of expression is known as **Product of Sum or POS** expression.

F1 in POS form can also be represented as

$$= M_0 \cdot M_2 \cdot M_3 \cdot M_5$$

Instead of using sum terms, we can just mention the maxterm designation.

$$= \pi(0,2,3,5)$$

This one is mathematical representation of the above expression.

CONVERT NON CANONICAL POS EXPRESSION TO CANONICAL POS EXPRESSION

Let's take one example:

Q Convert $(X+Y') \cdot (Y'+Z')$ into canonical SOP form.

Ans Steps to solve this problem

1. For a given expression, prepare a truth table for all possible combinations of inputs.
2. Add a new column for maxterms and list all the maxterms for all possible combinations.
3. Add all the maxterms for which the output is 0. This gives the canonical POS form.

X	Y	Z	Y'	Z'	X+Y'	Y'+Z'	(X+Y').(Y'+Z')
0	0	0	1	1	1	1	1
0	0	1	1	0	1	1	1
0	1	0	0	1	0	1	0
0	1	1	0	0	0	0	0
1	0	0	1	1	1	1	1
1	0	1	1	0	1	1	1
1	1	0	0	1	1	1	1
1	1	1	0	0	1	0	0

The canonical POS expression is:

$$F(x,y,z) = (x+y'+z) \cdot (x+y'+z') \cdot (x'+y'+z')$$

OR

$$= M_2 M_3 M_7$$

OR

$$= \pi(2,3,7)$$

To represent the function in complement form i.e. for its value 1, we will AND the maxterms, from the truth table, for which function value is 1. Thus the function in complement form is:

$$F'(x,y,z) = (x+y+z) \cdot (x+y+z') \cdot (x'+y+z) \cdot (x'+y+z') \cdot (x'+y'+z) \quad (\text{the remaining maxterms})$$

Let's see how to obtain SOP and POS expression for same function

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

SOP expression for F will be $x'y'z + x'yz'$ or $\sum (1,2)$

POS equivalent of it will be

$$= (x + y + z) \cdot (x + y' + z') \cdot (x' + y + z) \cdot (x' + y' + z') \cdot (x' + y' + z) \cdot (x' + y' + z')$$

OR

$$= \prod (0,3,4,5,6,7)$$

CONVERSION FROM SOP TO POS AND POS TO SOP

Conversion from one form to another is easy. Short cut for it is:

- Interchange the symbol (OR to AND / $\sum$ to $\prod$)
- Make a truth table and list those terms, which are missing in original.

MINIMIZATION OF BOOLEAN EXPRESSION

Till now we have studied canonical SOP and POS expressions. Now we will simplify these boolean expressions because simplified expressions are usually more efficient and effective when implemented in practice. Also simpler expressions are easier to understand, and results in simplified circuitry. So the canonical expressions are simplified/minimized to obtain a reduced expression (Standard form) which are then used for implementation of circuits in computers.

The two ways in which boolean expressions can be minimized are:

- i) Algebraic Method
- ii) Karnaugh Map

Algebraic reduction of Boolean function/expression:

In this method we make use of boolean postulates and theorems to simplify the expression.

Example 1:

$$\begin{aligned}
 F(x,y) &= (xy)' + x' + xy \\
 &= (x' + y') + x' + xy && \text{by De Morgan's theorem} \\
 &= x' + x' + y' + xy \\
 &= x' + xy + y' && \text{by Idempotent law} \\
 &= x' + y + y' && \text{by absorption law} \\
 &= x' + 1 && \text{by complementation law} \\
 &= 1 && \text{by NULL element}
 \end{aligned}$$

Example 2:

$$\begin{aligned}
 F(a,b) &= ab + ab' + a'b \\
 &= a(b+b') + a'b && \text{by Distribution law} \\
 &= a.1 + a'b && \text{by complement law} \\
 &= a + a'b && \text{by Identity} \\
 &= a+b && \text{by absorption law}
 \end{aligned}$$

Algebraic transformation, can always be used to produce optimal reduced form, but does not guarantee the reduced form. Also, it is not necessary to reach to same transformation, when worked on by different group of people. However, there are other methods using which, we will always reach to optimal solution every time.

Karnaugh Map (K-map)

A K-map is a graphical representation of truth table useful for reducing or minimizing Boolean expressions. Karnaugh map was developed by Maurice Karnaugh, at Bell Labs in 1953. The map reduces boolean expression more quickly and easily, as compared to algebraic reduction.

Steps to reduce boolean expression using K-map

1. Draw K-map
2. Plot K-map
3. Group the terms in minimum number of groups
4. Read the reduced expression from map

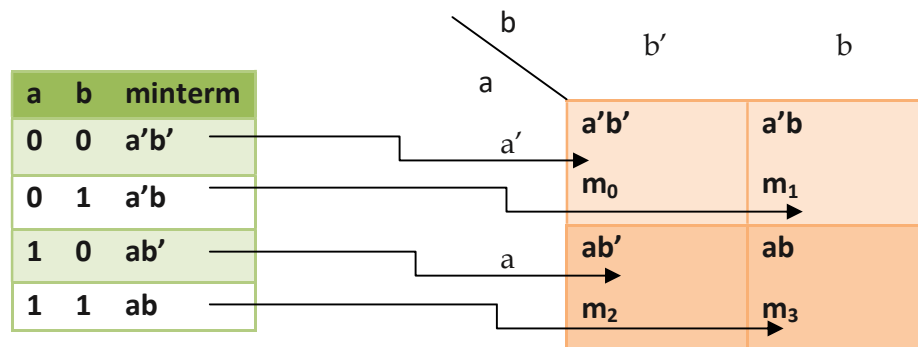
REDUCING SOP EXPRESSION USING K-MAP

DRAWING K-MAP

A K-map is a collection of squares, where each square represents one minterm in the case of SOP expression.

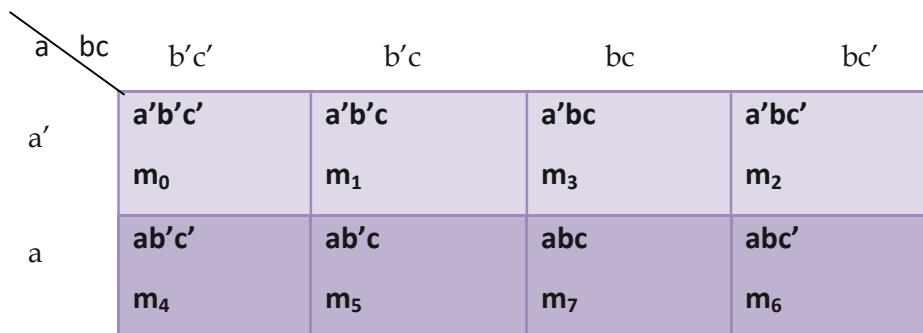
TWO VARIABLE K-MAP

In a two variable map there will be 4 squares. K-map diagram for two variables - a,b, will look like:



THREE VARIABLE K-MAP

In a three variables K-map, there will be 8 squares. K-map diagram for three variables - a,b,c will look like:



Or

	c	c'	c
ab			
a'b'	m ₀	m ₁	
a'b	m ₂	m ₃	
ab	m ₆	m ₇	
ab'	m ₄	m ₅	

FOUR VARIABLE K-MAP

In a four variables K-map, there will be 16 squares. K-map diagram for four variables - a,b,c,d will look like:

	cd	c'd'	c'd	cd	cd'
ab					
a'b'	m ₀	m ₁	m ₃	m ₂	
a'b	m ₄	m ₅	m ₇	m ₆	
ab	m ₁₂	m ₁₃	m ₁₅	m ₁₄	
ab'	m ₈	m ₉	m ₁₁	m ₁₀	

OR

	ab	a'b'	a'b	ab	ab'
cd					
c'd'	m ₀	m ₄	m ₁₂	m ₈	
c'd	m ₁	m ₅	m ₁₃	m ₉	
cd	m ₃	m ₇	m ₁₅	m ₁₁	
cd'					

m_2	m_6	m_{14}	m_{10}
-------	-------	----------	----------

Plotting the expression in the K-map

Once a K-map is drawn, next step is to plot the expression to be minimized to the K-map. **To plot the expression** it should be in canonical SOP form. For every minterm present in the boolean expression mark 1 in the corresponding cell of the K-map.

For example: consider the boolean expression

$$F(a,b,c) = a'b'c + ab'c + abc' + abc$$

Plotting of above boolean expression in the K-map

	bc	$b'c'$	$b'c$	bc'
a'	m_0	m_1	m_3	m_2
		1		
a	m_4	m_5	m_7	m_6
		1	1	1

GROUPING THE MINTERMS

Rules of grouping

1. A group must contain 1, 2, 4, 8, 16, cells, i.e. in the power of 2.
2. Groups can be horizontal or vertical, not diagonal.
3. Groups should not include any cell containing zero
4. In every group at least one 1 should be unpaired. The group in which no 1 is unpaired is called a redundant group and should be removed.
5. We need to group maximum number of 1's in minimum number of groups.
6. Groups can overlap.
7. Groups may wrap around the table. The left most cell(s) in a row may be grouped with rightmost cell(s) and the top cell(s) in a column may be grouped with the bottom cells(s). Cells in corners of the map are also adjacent and can be grouped.

CORRECT		INCORRECT	
		Group m_5, m_7 is a redundant group	

$a'b'$	cd	$c'd'$	$c'd$	cd'
	m_0	m_1	m_3	m_2
$a'b'$				1
$a'b$	m_4	m_5	m_7	m_6
ab	m_{12}	m_{13}	m_{15}	m_{14}
ab'	m_8	m_9	m_{11}	m_{10}
	1			

$a'b'$	cd	$c'd'$	$c'd$	cd'
	m_0	m_1	m_3	m_2
$a'b'$				1
$a'b$	m_4	m_5	m_7	m_6
ab	m_{12}	m_{13}	m_{15}	m_{14}
ab'	m_8	m_9	m_{11}	m_{10}
	1			

$a'b'$	cd	$c'd'$	$c'd$	cd'
	m_0	m_1	m_3	m_2
$a'b'$	1			1
$a'b$	1	m_5	m_7	1
ab	1	m_{13}	m_{15}	1
ab'	1	m_9	m_{11}	1

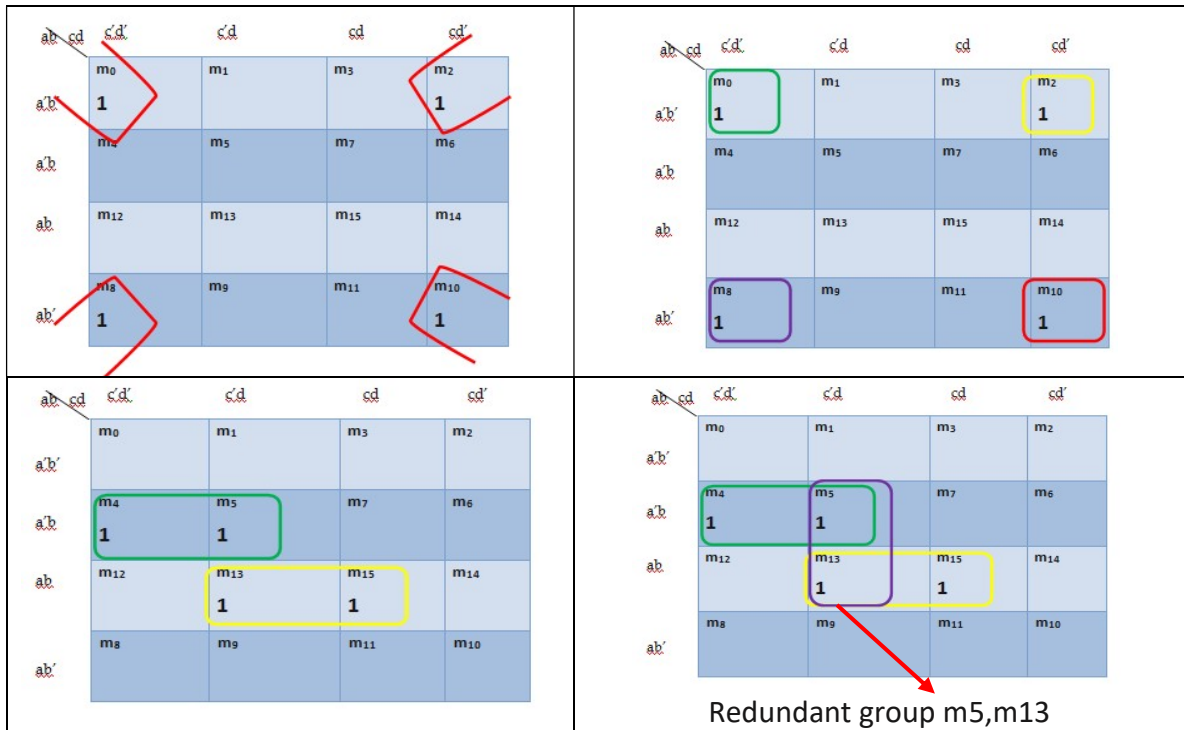
$a'b'$	cd	$c'd'$	$c'd$	cd'
	m_0	m_1	m_3	m_2
$a'b'$	1			1
$a'b$	1	m_5	m_7	1
ab	1	m_{13}	m_{15}	1
ab'	1	m_9	m_{11}	1

$a'b'$	cd	$c'd'$	$c'd$	cd'
	m_0	m_1	m_3	m_2
$a'b'$				
$a'b$	1	m_5	m_7	1
ab	1	m_{13}	m_{15}	1
ab'	m_8	m_9	m_{11}	m_{10}

$a'b'$	cd	$c'd'$	$c'd$	cd'
	m_0	m_1	m_3	m_2
$a'b'$				
$a'b$	1	m_5	m_7	1
ab	1	m_{13}	m_{15}	1
ab'	m_8	m_9	m_{11}	m_{10}

$a'b'$	cd	$c'd'$	$c'd$	cd'
	m_0	m_1	m_3	m_2
$a'b'$	1			1
$a'b$	1	m_5	m_7	1
ab	1	1	1	1
ab'	m_8	m_9	m_{11}	m_{10}

$a'b'$	cd	$c'd'$	$c'd$	cd'
	m_0	m_1	m_3	m_2
$a'b'$	1			1
$a'b$	1	m_5	m_7	1
ab	1	1	1	1
ab'	m_8	m_9	m_{11}	m_{10}



READING THE REDUCED EXPRESSION

Rules for writing the reduced expression

- 1) Each group corresponds to one product term, from K-map.
- 2) The term is determined by finding the common literals in that group, i.e. the variables which are present in both forms i.e. normal and complement are omitted in resultant product term.

K-map	SOP Expression
	$b'c + ac$

	$b'c + a$
	$c' + a$
	$a'b'cd' + ab'c'd'$
	d'
	bd'

	$ab + a'd'$
	$b'd'$
	$a'bc' + abd$

NOTE:

- ❖ **In a 3 variable K-map**
 - Group of 2 terms will have 2 boolean variables
 - Group of 4 terms will have 1 boolean variable
- ❖ **In a 4 variable K-map**
 - Group of 2 terms will have 3 boolean variables
 - Group of 4 terms will have 2 boolean variables
 - Group of 8 terms will have 1 boolean variable

REDUCING POS EXPRESSION

So far we were working with SOP expression/function. However, it is possible to reduce POS function also. In POS reduction, the approach is much the same except for, instead of minterms, we work with maxterms and we plot 0 instead of 1.

DRAWING K-MAP

THREE VARIABLE K-MAP FOR POS EXPRESSION

X \ YZ	Y+Z	Y+Z'	Y'+Z'	Y'+Z
X	M ₀	M ₁	M ₃	M ₂
X'	M ₄	M ₅	M ₇	M ₆

FOUR VARIABLE K-MAP FOR POS EXPRESSION

XY \ WZ	W+Z	W+Z'	W'+Z'	W'+Z
X+Y	M ₀	M ₁	M ₃	M ₂
X+Y'	M ₄	M ₅	M ₇	M ₆
X'+Y'	M ₁₂	M ₁₃	M ₁₅	M ₁₄
X'+Y	M ₈	M ₉	M ₁₁	M ₁₀

Plotting the expression in the K-map

Once a K-map is drawn, next step is to plot the POS expression to be minimized to the K-map. **To plot the expression** it should be in canonical POS form. For every maxterm present in the boolean expression mark 0 in the corresponding cell of the K-map.

For example: consider the boolean expression

$$F(a,b,c) = (a'+b'+c) \cdot (a+b'+c) \cdot (a+b+c') \cdot (a+b+c)$$

Plotting of above boolean expression in the K-map

a \ bc	b+ c	b+ c'	b'+ c'	b'+ c
a	M ₀ 0	M ₁ 0	M ₃	M ₂ 0
a'	M ₄	M ₅	M ₇	M ₆ 0

Rules of grouping and reading the reduced expression are the same as we discussed in the case of SOP expression.

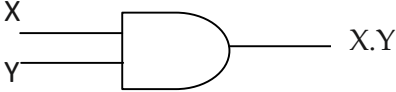
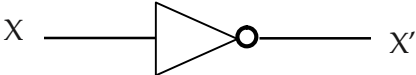
a \ bc	b+ c	b+ c'	b'+ c'	b'+ c
a	M ₀ 0	M ₁ 0	M ₃	M ₂ 0
a'	M ₄	M ₅	M ₇	M ₆ 0

The reduced POS expression is:

$$F(a,b,c) = (a+b).(b'+c)$$

LOGIC GATES

Logic gates are circuits that perform logical operations such as AND, OR and NOT. It is a block of hardware that takes binary signal(s) as input and produces a logic 0 or a logic 1 as output. There are three types of logic gates:

AND Logical Multiplication	
OR Logical Addition	
NOT Complementation	

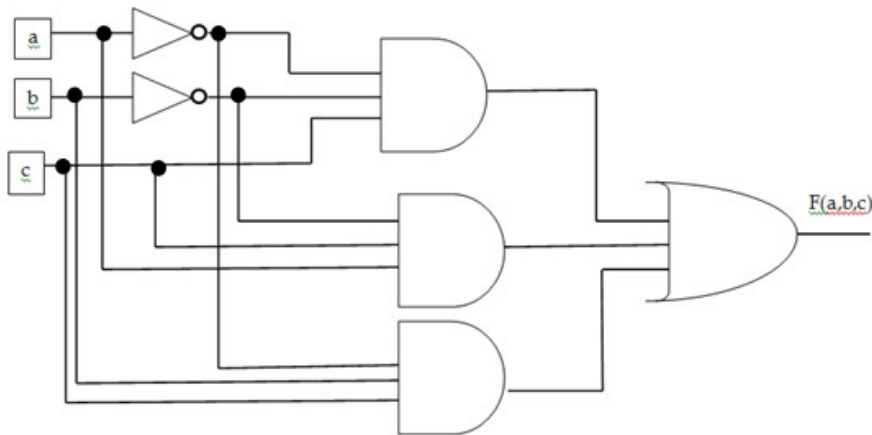
Gates can have more than two inputs except NOT gate.

REPRESENTING BOOLEAN EXPRESSION AS LOGIC GATE

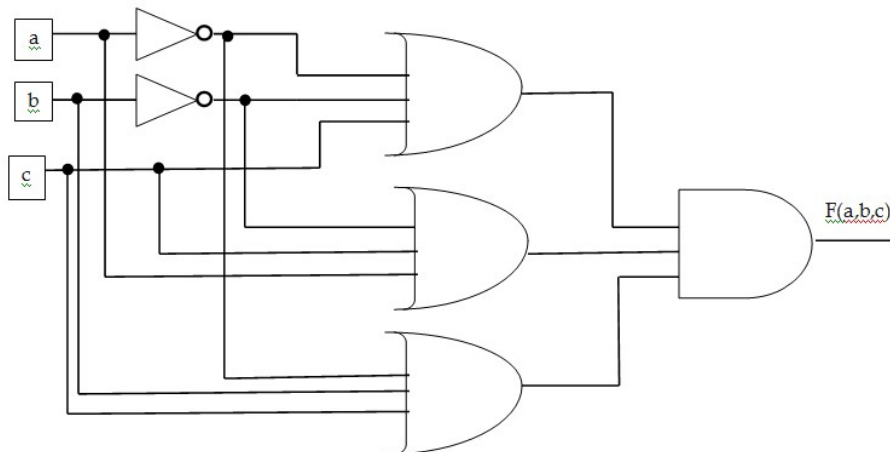
When a Boolean Function is implemented with logic gates, each literal in the function becomes an input to a gate.

Let's illustrate some Boolean expressions in circuit form.

$$F(a,b,c) = a'b'c + ab'c + a'bc$$



$$F(a,b,c) = (a' + b' + c) \cdot (a + b' + c) \cdot (a' + b + c)$$



NOTE: It is essential to minimize the boolean expression to reduce the total number of gates in the circuit.

UNIVERSAL GATES

The gates which can implement any boolean function without the help of any other gates are called Universal Gates.

The **NAND** and **NOR** gates are universal gates. Any logic circuit diagram can be realized using only these two gates.

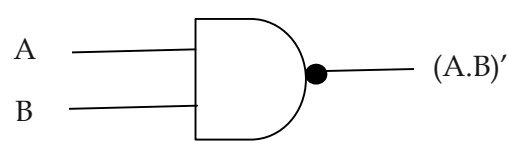
Advantage of Universal Gates

They are economical and easier to fabricate and are the basic **gates** used in all IC digital logic families.

NAND Gate:

For two variables A and B the truth table and logic circuit diagram of NAND gate is:

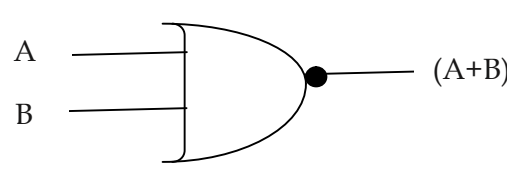
A	B	A.B	$(A.B)'/A \text{ NAND } B$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0



NOR GATE

For two variables A and B the truth table and logic circuit diagram of NOR gate is:

A	B	A+B	$(A+B)'/A \text{ NOR } B$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

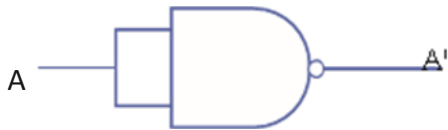


Now let's see how each logic gate can be created with universal gate, so that entire circuit can be implemented through single type of gate.

REALIZATION OF ALL GATES USING NAND GATE

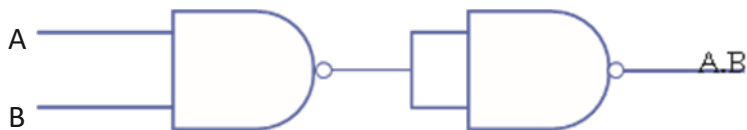
NOT GATE

$$\begin{aligned} A' &= (A.A)' \\ &= A \text{ NAND } A \end{aligned}$$



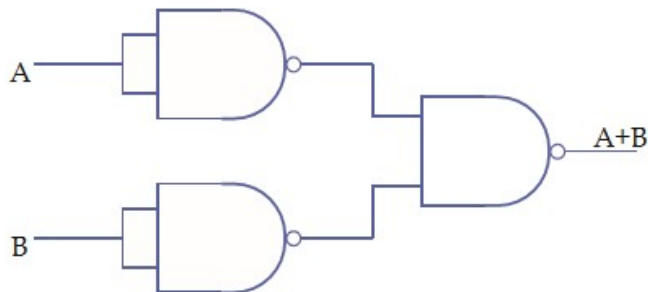
AND GATE

$$\begin{aligned} A.B &= ((A.B)')' \\ &= (A \text{ NAND } B) \text{ NAND } (A \text{ NAND } B) \end{aligned}$$



OR GATE

$$\begin{aligned} A+B &= ((A+B)')' \\ &= (A'.B')' \\ &= ((A.A)'.(B.B)') \\ &= (A \text{ NAND } A) \text{ NAND } (B \text{ NAND } B) \end{aligned}$$



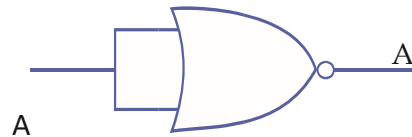
REALIZATION OF ALL GATES USING NOR GATE

NOT GATE

$$= (A)'$$

$$= (A+A)'$$

i.e. A NOR A



AND GATE

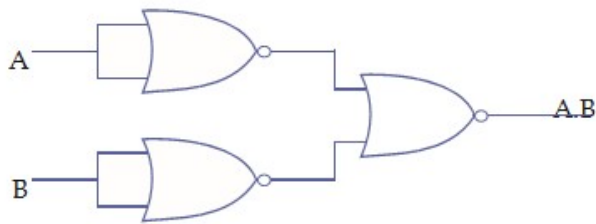
$$= A.B$$

$$= ((A.B))'$$

$$= (A'+B)'$$

$$= ((A \text{ NOR } A) + (B \text{ NOR } B))'$$

i.e. (A NOR A) NOR (B NOR B)

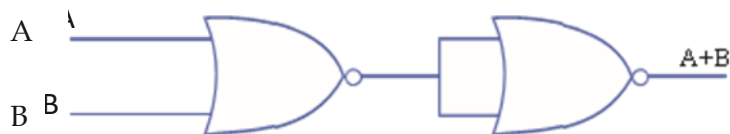


OR GATE

$$= A+B$$

$$= ((A+B))' = (A \text{ NOR } B)'$$

i.e. (A NOR B) NOR (A NOR B)



IMPLEMENTING COMPLETE CIRCUIT THROUGH UNIVERSAL GATE

RULES:

1. To implement the complete circuit using NAND gates the expression should be in SOP form and to implement using NOR gates the expression should be in POS form.
2. Take the double complement of the expression(By law of Involution)
3. Solve the inner complement using De Morgan's law.

EXAMPLES

NAND GATE

Q. Implement the following circuit through NAND universal gates.

$$F(a, b, c, d) = ab'c + b'cd + bcd$$

The above expression is in SOP form. Now take the double complement of the expression and solve the inner one.

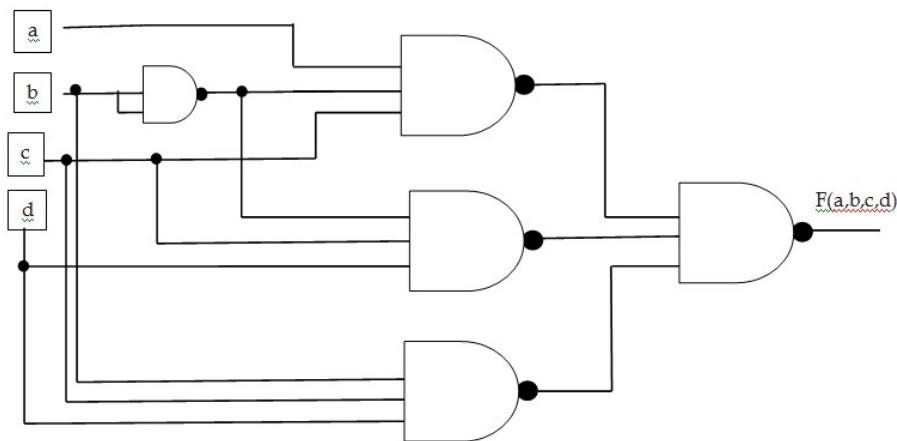
$$= ((ab'c + b'cd + bcd)')'$$

Now solve the inner complement using De Morgan's law

$$= ((ab'c)' \cdot (b'cd)' \cdot (bcd)')'$$

$$= (a \text{ NAND } (b \text{ NAND } b) \text{ NAND } c) \text{ NAND } ((b \text{ NAND } b) \text{ NAND } c \text{ NAND } d) \text{ NAND } (b \text{ NAND } c \text{ NAND } d)$$

So now the complete circuit may be represented using NAND gate as:



NOR GATE

Q Implement the following circuit using NOR gate(s)

$$F(a, b, c, d) = (a' + b + c)(a + b + c)(b + c + d)$$

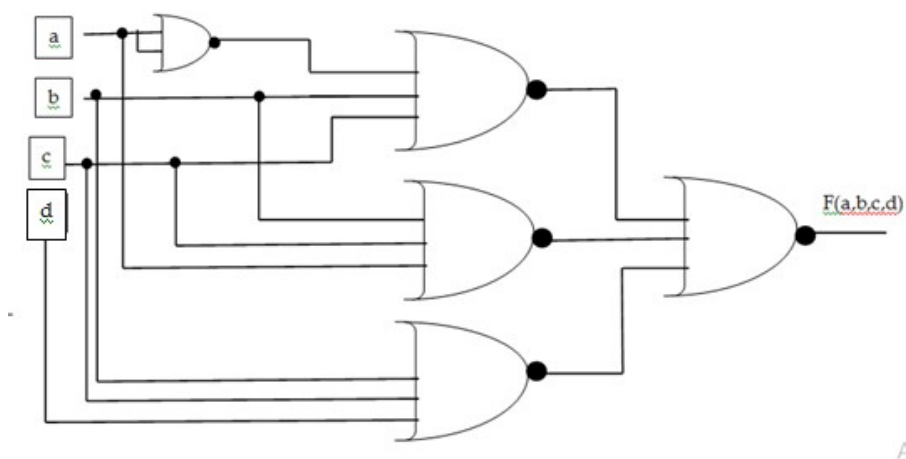
The above expression is in POS form. Now take the double complement of the expression and solve the inner one.

$$= ((a' + b + c)(a + b + c)(b + c + d))''$$

$$= ((a' + b + c)' + (a + b + c)' + (b + c + d)')'$$

$$= ((a \text{ NOR } a) \text{ NOR } b \text{ NOR } c) \text{ NOR } (a \text{ NOR } b \text{ NOR } c) \text{ NOR } (b \text{ NOR } c \text{ NOR } d)$$

So now the complete circuit may be represented using NOR gate as:



One of the most recent uses of Boolean logic is, Internet search engine and Database search. Internet is a vast computer database and the proper use of Boolean operators - AND, OR and NOT would increase the effectiveness of web search. These operators can act as effective filters for finding just the information one need from Internet. So most of the search engine support some form of Boolean query using Boolean operators.

